REPORT
Phishing Attack Simulation and Mitigation

Introduction

Today we see the rise in technology more than ever before and with that we've now begun to see cybersecurity threats become more complex and happening more often than ever before. In this report we will show how phishing attacks are some of the most common and dangerous ones. Phishing is a form of cyber attack where the bad actors imitate legitimate organizations with the purpose of tricking people into giving up valuable information. This can be anything from passwords, money, or personal details.

Phishing attacks rely on social engineering, oftentimes not even using technology to execute the attack. Attackers look for different ways they can exploit people's trust, urgency and curiosity. This leads to a phisher using fear to manipulate their victims. Now that we're seeing unprecedented technological advances, phishing campaigns are now becoming more complex and difficult to spot. It's gotten to the point that individuals and organizations must stay up to date with these threats and participate in awareness training to understand the threats.

The history and evolution of phishing

Looking at the history of phishing we can see the first wave of phishing attacks began in the 1990s. Back then with the internet being relatively new and not as widespread, bad actors used email and instant messaging to trick people into giving up access to their accounts. Also important to note, the term "phishing" comes from the word "fishing," which came about due to the idea of luring people in. Early phishing attempts were actually quite simple, relying on sending mass emails. But, as time has passed, bad actors have improved their methods, which now include using fake sites and creating programs to help their scams look real.

As internet access became more common and more and more services moved online, the rate of phishing attacks grew significantly. This is shown by the early 2000s, where phishing became world wide. Furthermore with the rise of social media and mobile apps, phishing was further increased in scope and scale. Today, attackers use automation,AI, and a large amount of data gathering to create these highly complex phishing messages that have the ability to evade traditional security measures.

Assault Mechanisms and Vulnerabilities

Phishing attacks use different modes of digital communication and vulnerabilities. The most common ones used for the attacks include: Email Phishing, which is the sending of bulk emails claiming to be from already established organizations, such as, banks. These emails use things like scare tactics to instill fear in the victim and contain malicious links that, once opened, leads to either a fake site or downloads malware. Another type of phishing attack is called smishing, which is a form of phishing that uses text messages to send false info. Attackers text out links to fake websites or prompt users to call fake customer service numbers all for the purpose of stealing information. Vishing is a method of voice phishing where attackers call people, pretending to be a representative of legitimate organizations, to ask for details.

Phishing can be found anywhere that media is present like Facebook, Twitter, and LinkedIn, which are used to send phishing messages. These methods take advantage of people and their common tendencies, most specifically the tendency to trust something that is familiar to them, including a recognizable name, or form of communication. Even more, since we have transitioned into an increased use of remote work, this greatly expands the scope of vulnerability to phishing attacks. Oftentimes we see employers require, work-from-home employees, to use security measures such as VPN’s and train them to use the principle of zero trust in order to avoid any potential attacks.

Types of Phishing Attacks

In 2010, computer security specialist Jason Hong outlined several different types of phishing in "The State of Phishing Attacks." Here he explains that there are three different types of attacks which are spear phishing, whaling and clone phishing.

Spear phishing is a form of phishing campaign that targets a specific person or group and often will include information known to be of interest to the target, such as current events or financial documents. An example is when a phishing e-mail is designed to look like it is from a potential job opportunity after sending out applications online.

Whaling is a common cyber attack that occurs when an attacker utilizes spear phishing methods to go after a large, high-profile target, such as executives. Whaling attacks may come in the form of legal documents or wire transfer requests.

Clone phishing is a newer type of email-based threat where attackers clone a real email message and resend it pretending to be the original sender. Any attachments are replaced with malware. The similarity of the emails increase the likelihood of user interaction, often you'll see the emails are indistinguishable from each other so it's best to double check the email address.

Consequences of Phishing

Recent reports have detailed a surge in both the occurrence and severity of phishing attacks. The Anti-Phishing Working Group reported nearly 989,000 cases of phishing attacks on its member companies for just the last quarter of 2024 alone, which is one of the highest ever recorded levels. There are rough estimates that say over 3.4 billion phishing emails are done globally per day. Financial losses that happened as a result of phishing attacks have crossed into the billions of dollars mark, with people in the United States alone reporting $16 billion lost to cyber fraud during 2024, a 33% increase on the previous year.

The reports show that the effectiveness and low costs involved for phishers when they launch their attacks, combined with the amount of money they bring in, makes this scam very lucrative for them. Unlike other computer attacks that require knowledge and skill in finding/exploiting vulnerabilities from software or hardware, phishing requires little resources to start and yet has huge outcomes.

Psychological manipulation and social engineering

Phishing attacks often succeed not due to some technological ability but because of social engineering, which is manipulating human behavior to gain unauthorized access to systems or data. There are several psychological principles attacks may exploit in phishing which can include:

Urgency: which is seen in messages that urge people to act quickly, an example of this would be extending your car's warranty before it expires.

Authority: which would be the impersonation of authority figures, such as a CEO or bank representative, in order to scare people into complying.

Fear and Threat: this tactic works by scaring users with false consequences like account termination or legal action.

Curiosity and Incentive: this works by luring people in with things like rewards, promotions, or money to motivate people to click on harmful links.

By using the power of the human condition, attackers increase the chances that their messages will be opened and acted upon. Studies have shown that phishing attacks that use this sense of urgency and harm have success rates that can be up to 20% higher than that of a regular phishing email.

Advancements in technology and strategy building

Modern phishing activities have become more complex largely due to the evolution of technology:

Artificial intelligence has been used in many ways to enhance phishing attempts, this includes realistic looking phishing emails, social engineering attempts tailored to the victim, and also the ability to generate fake voices for voice phishing campaigns.

There's also now something called a phishing kit, which are pre built tools that allow people with little to no technical knowledge to launch a full blown phishing attack, you can do anything from fake login screens to storage systems of people's information.

Next we have URL obfuscation and domain spoofing. These techniques are used to make malicious URLs look authentic. One such example of this are homograph attacks which would do something like replacing 'o' with 'o' from Cyrillic thus tricking the user into believing it is the official site.

HTTPS Abuse: A large percentage of phishing sites now use SSL certificates to display the "secure" padlock icon, fooling users into trusting them.

Such patterns emphasize the requirement for constant watchfulness and innovative thinking when creating defense strategies.

Preventive measures and optimal strategies

With phishing attacks growing more sophisticated, a two-pronged approach that includes user education and technological measures is the best method of mitigation. Some of the most effective techniques include:

Multi-Factor Authentication (MFA): this works by asking the user to have multiple forms of verification, such as a password and a one time passcode sent to your phone. This extra step makes it much harder for attackers to get access to accounts, even if they are able to figure out the username and password. Studies show MFA reduces successful phishing attempts by over 90%.

User training and education: Regular training sessions that help users identify and report suspicious emails help decrease the click-through rates of phishing links.

Email filters and anti-phishing software: We used something similar for our project, which we will expand on further below. But these are basically tools that screen and eliminate potential phish emails before they reach the users' inbox.

Incident Response Plans: This is effective because it establishes procedures for responding to phishing attempts, which include isolating any impacted systems and bringing it to the attention of managers.

Legal, ethical, and societal implications of phishing

Phishing is not only a major cyber threat but also raises far-reaching legal and ethical issues for individuals, organizations, and society at large. Legally, phishing is criminalizable across different jurisdictions under a number of fraud, privacy, and cybersecurity laws. In the United States, for example, the Computer Fraud and Abuse Act, as well as the Identity Theft and Assumption Deterrence Act, provides for the prosecution of perpetrators who engage in misleading schemes for the unauthorized collection of people's personal information. Likewise, international law, typified by the European Union's General Data Protection Regulation, levies serious sanctions on organizations that do not sufficiently prevent or respond to phishing attacks that compromise user details.

In ethical terms, phishing breaks down user trust and discourages people from using online services. On a business level, the ethical implications are even worse, if a company fails to alert their employees or protect their customers information against phishing, they greatly increase the risk of financial loss as well as a loss of reputation and in the event of a more severe attack potential legal consequences.

Spam Email Detection System

The provided code implements a spam email detection system that combines modern machine learning techniques with email protocol integration. This system is designed to connect to an email server, analyze incoming messages, identify potential spam based on trained models, and take appropriate actions like moving spam to trash. The solution leverages natural language processing, the Naive Bayes classification algorithm, and email handling libraries to create an end-to-end spam filtering system. The implementation represents a practical application of text classification in cybersecurity, specifically addressing the common problem of email spam filtering. This report will analyze each component of the code in detail, explaining the underlying principles, implementation choices, and potential improvements.

System Architecture

The core of the system is the SpamEmailDetector class, which encapsulates all functionality required for spam detection and email management. The class follows an object-oriented design pattern with clear separation of concerns across its methods. The overall architecture can be divided into four main components: methods that handle email text extraction and normalization, train and apply the classification model, connect to email servers and retrieve messages, and identify and handle detected spam.

The preprocess_text method implements a series of text normalization steps to prepare raw email content for machine learning. This method applies several key transformations to standardize the text. First, it converts all text to lowercase, ensuring consistent case throughout the text and preventing duplicate features like "Money" and "money" from being treated differently. Next, it strips HTML markup that could interfere with content analysis by removing tags that aren't relevant to the actual message content. The method then focuses analysis on word content by replacing numbers, punctuation, and special characters with spaces, emphasizing the textual elements most useful for classification. Finally, it removes redundant spaces to standardize the text format, creating clean input for the machine learning algorithms. These preprocessing steps are crucial for reducing noise in the data and focusing the model on meaningful textual patterns that distinguish spam from legitimate emails.

```
def preprocess_text(self, text):
    """Clean and normalize text"""
    # Convert to lowercase
    text = text.lower()
    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)
    # Remove non-letters
    text = re.sub(r'[^a-z\s]', ' ', text)
    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

The train method implements the supervised learning process. This method handles the entire training workflow for the spam classification model. It begins by reading email files from separate spam and ham directories, carefully organizing the dataset structure. The system then adds preprocessed emails and their corresponding labels to lists, marking spam as 1 and legitimate emails (ham) as 0. To ensure proper evaluation, the dataset is divided into training (80%) and testing (20%) subsets using scikit-learn's train_test_split function. For feature extraction, the method uses CountVectorizer to convert email text into numerical features through a document-term matrix, where each row represents an email, each column represents a specific word, and values indicate word frequencies. The Multinomial Naive Bayes classifier is then trained on this feature matrix, learning the statistical patterns that differentiate spam from legitimate emails.

The predict method applies the trained model to new emails. This method performs the crucial task of analyzing new emails for spam content. It begins with validation to ensure the model has been trained before attempting prediction, preventing potential errors from untrained models. The method then applies the same preprocessing steps used during training to maintain consistency across the pipeline. For feature extraction, it transforms the preprocessed text into a feature vector using the trained vectorizer, enabling the classifier to analyze the email using the same vocabulary and feature space as during training. The classifier makes a binary prediction (spam or not) and retrieves the probability estimate, providing both a definitive classification and a measure of confidence. Finally, the method returns a dictionary with the boolean classification and confidence score, offering valuable information for decision-making when the classification might be uncertain. This approach allows the system to potentially implement confidence thresholds or apply different actions based on the certainty of the spam classification.

The connect_to_email method establishes a connection to an email server. This method handles the essential task of connecting to an email service to access messages for analysis. It creates a secure IMAP connection using SSL, which ensures encrypted communication with the email server. The method then authenticates with the provided credentials, establishing an authenticated session with the email service. After successful authentication, it selects the specified mailbox (defaulting to "INBOX") to determine which messages will be available for scanning. Upon successful completion, the method returns the connection object that will be

used by other methods to retrieve and manipulate emails. If any errors occur during this process, the method safely captures and reports authentication or connection issues, returning None to indicate the failure. The default parameters are set for Gmail, but the method can be used with any IMAP server by overriding the defaults, making it flexible for various email providers.

```
def connect_to_email(self, email_address, password, imap_server="imap.gmail.com", ma
    """Connect to an email server via IMAP"""
    try:
        # Connect to the server
        mail = imaplib.IMAP4_SSL(imap_server)
        mail.login(email_address, password)
        mail.select(mailbox)
        return mail
    except Exception as e:
        print(f"Error connecting to email: {e}")
        return None
```

The scan_emails method retrieves and analyzes emails. This method performs the core function of identifying spam emails in the user's inbox. It begins with validation to ensure the model is trained before attempting to analyze any emails. The method then searches for all emails in the selected mailbox and retrieves their message IDs, creating a comprehensive list of available messages. To manage processing load and focus on the most relevant emails, it limits processing to the most recent messages, controlled by the limit parameter, which defaults to scanning the 10 most recent emails. The system then processes each selected email individually by fetching the full email content, parsing it into a message object, and extracting and decoding the subject line to provide identifiable information about each message. It extracts the text content using the previously defined extraction method and applies the spam prediction model to classify the email. When an email is classified as spam, the method collects its ID, subject, and confidence score in a result list that will be used for reporting and potential email management actions. The method prioritizes recent emails by slicing the message ID list from the end, which is a practical approach for ongoing spam monitoring in active email accounts.

```python
def remove_spam(self, mail_connection, spam_emails, move_to_trash=True):
    """Remove or move spam emails to trash"""
    try:
        for spam in spam_emails:
            if move_to_trash:
                # Move to trash (Gmail's "[Gmail]/Trash" folder)
                mail_connection.store(spam['id'], '+X-GM-LABELS', '\\Trash')
                mail_connection.expunge()
            else:
                # Delete permanently
                mail_connection.store(spam['id'], '+FLAGS', '\\Deleted')
                mail_connection.expunge()

        return True

    except Exception as e:
        print(f"Error removing spam: {e}")
        return False
```

The remove_spam method takes action on identified spam. This method iterates through the list of identified spam emails. If move_to_trash is True, it moves emails to the Gmail trash folder using Gmail-specific IMAP commands.

This execution flow involves command-line argument handling, expecting a path to training data. It continues with object initialization by creating a detector instance, followed by model training that processes the provided dataset. The email integration component connects to a Gmail account using credentials, after which spam detection scans for spam in the most recent 20 emails. The system then moves to result reporting, displaying identified spam with confidence scores. Spam handling moves identified spam to trash, though the confirmation prompt is commented out in the code. Finally, cleanup occurs with proper logout from the email server.

```
# Initialize the detector
detector = SpamEmailDetector()

# Train the model
print("Training the model...")
success = detector.train(data_directory)

if success:
    # Connect to email
    print("Connecting to email...")
    email_address = "josephjohnny263@gmail.com"  # Replace with your email
    password = "wfzy lygp bpup piqu"  # Replace with your password or app passwo

    mail = detector.connect_to_email(email_address, password)
    if mail:
        print("Scanning for spam...")
        spam_emails = detector.scan_emails(mail, limit=20)

        if spam_emails:
            print(f"Found {len(spam_emails)} spam emails:")
            for spam in spam_emails:
                print(f"  - {spam['subject']} (Confidence: {spam['confidence']:.

            # Ask for confirmation before removing
            #confirm = input("Do you want to move these emails to trash? (y/n):
            #if confirm.lower() == 'y':
            detector.remove_spam(mail, spam_emails)
            print("Spam emails moved to trash.")
```

Several potential improvements could enhance the system. Feature engineering could expand beyond the simple bag-of-words model to include email metadata, URL counts, or specific spam-related patterns to improve accuracy. Advanced models beyond Naive Bayes, such as Support Vector Machines, Random Forests, or modern deep learning approaches, could provide better classification results. The implementation would benefit from hyperparameter tuning for the classifier to potentially improve performance. Implementing incremental learning would allow the model to adapt over time as new spam patterns emerge without requiring retraining from scratch. Better security practices should replace hard coded email credentials with environment variables or properly secured configuration files. Adding a graphical user interface would make the system more accessible to non-technical users, and implementing a scheduling mechanism would enable periodic execution without manual intervention.

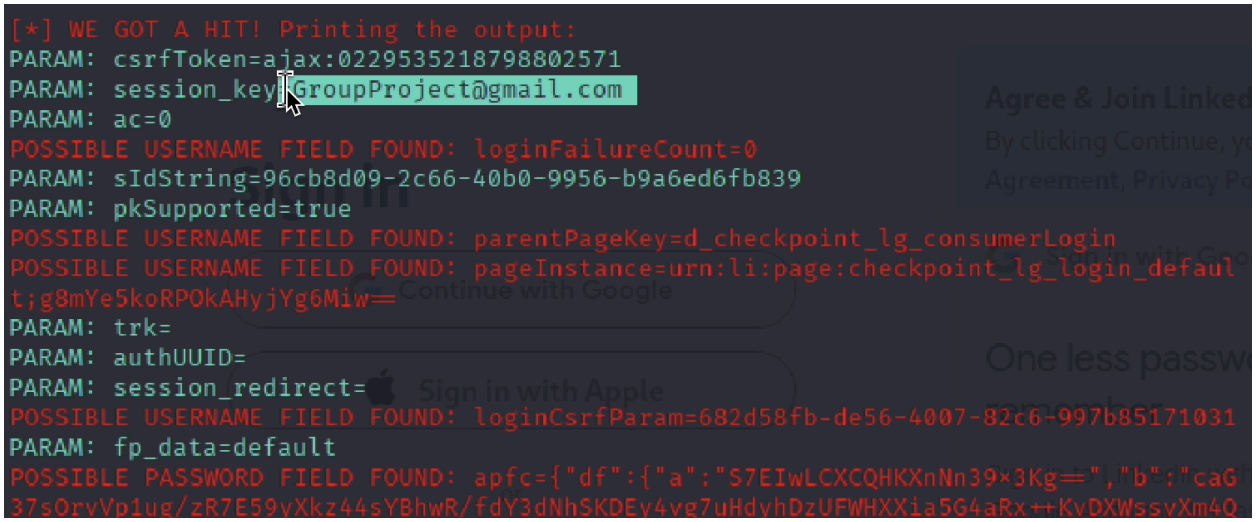
SpamEmailDetector provides a comprehensive solution for email spam filtering, combining text processing, machine learning, and email integration in a well-structured Python class. While

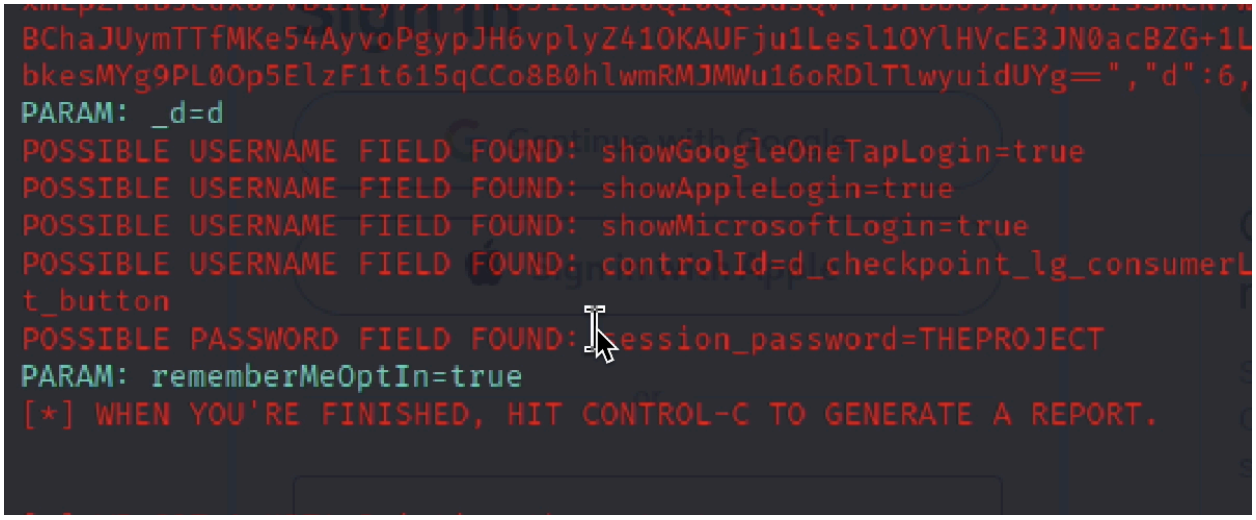
there are areas for improvement, particularly around security practices and model sophistication, the implementation demonstrates good software engineering principles and practical application of machine learning concepts.

Network topology, Configuration, and system Setup

For the Phishing attack, multiple setups were done ˇhe first thing done was downloading the Kali linux virtual machine which is basically use as a hacking tool. In the set up many extensions were downloaded to it. The second thing done was finding an actual website that we wanted to replicate. The replication of the website had to be the same, no difference so that the user won't be able to tell a difference in it. In order for us to do this , we Grabbed the website and looked at its source code. When analyzing it, We noticed that the login information was being sent to their servers which is what we wanted. The goal was to get the users credentials, But instead of sending it to the actual official server. We wanted it to be send to us,

As we can see from the code, we changed the action part which is getting all the credentials sent to the ip address . From this we were able to get all the credentials from the user without them noticing due to the fact that as soon as they wrote their credentials, the fake website would just refresh and send them back to the original one. This made them think the website had a glitch and that they would have to re enter their credential again.





From analyzing these two photos of what the results look like we can see the result. We can see that the password input was = THE PROJECT, And the username that the user input was GroupProject@gmail.com . We can see the attack was successful after getting the users credentials and we made sure the surname didn't even think about the website being fake , since it was literally the same set up as the original one, with exact buttons and the same display to create confusion.

A countermeasure for this phishing attack:

The group would recommend immediate action, The most immediate one would be to block the IP address or if it's a domain being used. You should attempt to conduct an analysis on the affected system or device to ensure which areas are affected and which ones are not. We would recommend making a reset to the password for any account. A more advanced option to prevent future attacks is we would recommend enhancing the email filtering so that you can filter any unwanted emails. Another option would be having an anti phishing software to detect phishing attempts and be able to reduce false positives. If it's an organization make sure you implement a multi factor authentication for every user account. It is very important to also have preventive training for users to be able to identify a phishing attack or have some knowledge on what to do in case of these attacks. For the implementation part we would recommend to have someone or a team to monitor incoming network traffic, Create rules and update them regularly And just overall maintain a response protocol for swift action to mitigate further risks and have a safer environment

.

Conclusion:

The phishing attack simulation showed us the danger and the effectiveness of phishing techniques in obtaining user credentials. The analysis highlighted the critical vulnerabilities exploited including user trust and the manipulation of login forms In conclusion after showing all the procedure and and risks we provided long-term countermeasures to enhance system security

and user awareness. This experiment emphasizes the importance of continuous monitoring and user education in mitigating phishing risks. The most important thing is that people should be aware and have knowledge of these sorts of attacks so we can mitigate them and just overall prevent credentials hijacking.

References

Bhowmick, Alexy & Hazarika, Shyamanta. (2018). E-Mail Spam Filtering: A Review of Techniques and Trends. 10.1007/978-981-10-4765-7_61.

https://www.linkedin.com/login

https://www.microsoft.com/en-us/security/business/security-101/what-is-phishing?ef_id=_k_Cj0KCQjwiqbBBhCAARIsAJSfZkZ2vGxh5Q2VVjaaVNQ6CyHDZIZa1-Z3hG4ZL5pRSCx98SqYBbm98MUaApxNEALw_wcB_k_&OCID=AIDcmmdamuj0pc_SEM__k_Cj0KCQjwiqbBBhCAARIsAJSfZkZ2vGxh5Q2VVjaaVNQ6CyHDZIZa1-Z3hG4ZL5pRSCx98SqYBbm98MUaApxNEALw_wcB_k_&gad_source=1&gad_campaignid=14495145267&gbraid=0AAAADcJh_tamWCNuGnWl_quaClxo6xIr&gclid=Cj0KCQjwiqbBBhCAARIsAJSfZkZ2vGxh5Q2VVjaaVNQ6CyHDZIZa1-Z3hG4ZL5pRSCx98SqYBbm98MUaApxNEALw_wcB

https://www.ncsc.gov.uk/guidance/phishing